

Knowledge Distillation: 把强模型的能力蒸馏进可控的 student LLM

图文讲义版：用原 PPT 作为视觉锚点，按“概念故事线 + 直觉解释 + 考试速记”整理。

现实矛盾：proprietary LLM 强、更新快、鲁棒，但黑盒、贵、有隐私风险 → 目标对象：open-source/student LLM 可控、便宜、可本地部署，但性能和 instruction coverage 往往较弱 → 核心想法：让强 teacher 产生可训练的 distillation knowledge，再继续训练 student → 总体 recipe：确定目标 skill/domain 与 template，准备 seed knowledge，让 teacher 生成知识，再训练 student → 数据公式：template I 与 seed S 经过 teacher 分布 p_T 生成输出 o，再由 Parse 变成 D_I^k → 获取知识一：Labelling、Expansion、Data curation 主要靠 black-box teacher 生成输入输出样本 → 获取知识二：Feature、Feedback、Self-knowledge 分别使用内部表示、teacher 评价、模型自我迭代 → 迁移知识：SFT 学答案, divergence/similarity 学分布或表示, RL/DPO 学偏好反馈 → 考试主线：区分 black-box/white-box、六种 knowledge extraction、四种 transfer methods，以及它们各自的代价和适用场景

Proprietary large language models

Disadvantages:

- Provided as a blackbox API - no way of checking the activations directly
- Issues with privacy (sending data to external services)
- High cost incurred (charged “per token”)

这节课一句话

Knowledge Distillation 在 LLM 时代不是单纯“把大模型压小”，而是用 teacher LLM 生成标签、数据、反馈、偏好或内部知识，再用 SFT、分布/表示对齐、RL 或 DPO 等训练方式，把昂贵黑盒模型的能力迁移到更便宜、更可控、更易部署的 student model。

1. 为什么需要 KD: 强 API 与可控 student 之间的桥

Proprietary large language models

Disadvantages:

- Provided as a blackbox API - no way of checking the activations directly
- Issues with privacy (sending data to external services)
- High cost incurred (charged “per token”)

人话直觉

把 proprietary LLM 想成全校最强但很贵的家教：它知识广、更新快、答题稳，可是你只能隔着电话问它；它怎么想你看不见，每问一句都收费，还可能要把敏感题目发到外部平台。Open-source LLM 像你自己家的学生：不一定最强，但你能看作业、改训练、部署在本地，也不用每次交 token 费。KD 的目标就是让强家教不只是替你答题，而是把做题方法尽量教给你自己的学生。

精确定义 / 机制：Lecture 10 的开场先对比 proprietary 与 open-source LLM。Proprietary LLM 的优势是 robust、能处理多类 query、持续更新、性能高；缺点是 black-box API、难以检查 activations 或内部表示、有 privacy issue，并且按 token 收费。Open-source LLM 的优势是可以控制模型和 representations、成本较低、能本地运行、隐私风险小，训练来源有时更透明；缺点是性能通常低于顶级专有模型、fine-tuned tasks/instructions 较少，本地服务也可能更难维护。Knowledge Distillation 正是卡在这组矛盾中间：用高性能 teacher LLM 生成知识或监督信号，再训练 student/open-source model，以缩小性能差距，同时保留低成本、可控和可部署的优势。

考试问法：如果考“为什么从 proprietary LLM 蒸馏到 open-source LLM”，不要只说“因为便宜”。高分答案要成对比较：teacher/proprietary 强但黑盒、贵、有隐私风险；student/open-source 可控、可本地部署但能力较弱；KD 试图把 teacher 的能力转移到 student，以获得 cost、privacy、accessibility 和 deployment 的综合收益。

2. KD 的一句话定义：teacher 不是终点，student 才是产物

Knowledge distillation

Main idea: Use the highly performing LLMs (OpenAI's models, for example) to further train open source models (such as Llama) to improve the latter's performance

Analogy: the big LLMs are the "teachers" and the open source models are the "students"

人话直觉

很多人会把 KD 误解成“以后答题都去问大模型”。其实这不是 distillation，这是 outsourcing。真正的 distillation 像酿酒：teacher 的输出、解释、偏好或特征只是原料；经过筛选和训练之后，能力要沉进 student 参数里。考试时问 KD，核心词一定是 teacher、student、generated knowledge、further training。

精确定义 / 机制：Knowledge Distillation 在 LLM 语境中指：用高性能 teacher LLM，例如强 proprietary model，产生数据、标签、解释、反馈、偏好或更细粒度的知识，然后用这些知识进一步训练较小、较便宜或开源的 student model。它在 LLM 中有三类角色：第一，主要用于 capability enhancement，让 student 学会 teacher 的任务能力或 instruction-following；第二，保留传统 compression 目的，提高效率、降低部署成本；第三，支持 self-improvement，即模型通过自生成知识迭代增强自己。需要注意，LLM KD 不一定要求访问 teacher 内部参数，许多方法只需要 black-box API 输出；但 feature、divergence、similarity 这类方法通常需要 white-box access。

考试问法：如果考“what is Knowledge Distillation in LLMs”，答法可以是：A teacher LLM produces synthetic data, labels, feedback, preferences or distributions, and a student LLM is further trained on this knowledge to acquire capabilities at lower cost or with better controllability. 记得指出它不只是 compression，也包括能力增强、instruction tuning、domain adaptation 和 self-improvement。

3. KD Recipe: 四步流水线，把 teacher 输出变成训练资产

Recipe for KD

- Target skill or domain in the teacher LLM with template I
- Seed knowledge as input: feed the teacher LLM with seed knowledge (S) - can be embodied in various ways (demonstrations, existing data, meta-knowledge)
- Generation of distillation knowledge: use teacher LLM to generate knowledge examples
- Training the student model with the generated knowledge

人话直觉

做 KD 不是随便打开 ChatGPT 让它吐一堆答案。它更像搭一条教学数据工厂：先决定要教什么课，写好老师的出题模板，准备种子材料，再让老师批量生成可训练的样本。最容易翻车的不是公式，而是工厂原料脏、模板歪、解析错，最后学生把垃圾也学进去了。

精确定义 / 机制：讲义给出的 KD recipe 有四阶段。第一，target skill or domain in the teacher LLM with template I ，即先决定要蒸馏的能力、领域和 prompt/instruction 模板。第二，seed knowledge S 作为输入，可以是 demonstrations、existing data 或 meta-knowledge。第三，generation of distillation knowledge，让 teacher LLM 根据 template 与 seed 生成知识样本。第四，training the student model with the generated knowledge，用标准训练目标进一步训练 student。这个 recipe 强调：KD 的核心 trick 是从更强模型中抽取可训练知识，而不是神秘的新模型结构；真正影响效果的是 template 设计、seed 覆盖、teacher 输出质量、解析与过滤、以及 student 训练细节。

考试问法：如果考“describe the KD recipe”，按四步答最稳：choose target skill/domain and template I ；prepare seed knowledge S ；use teacher to generate distillation knowledge；train student on generated knowledge。可补一句：data quality control 是关键风险，因为 teacher 的错误、偏见、重复和 hallucination 会被 student 学到。

4. 核心公式: D_I^{kd} 是怎样从 prompt 和 seed 里长出来的

Recipe for KD

These four stages can be represented using two stages:

Get the data: $D_I^{kd} = \{\text{Parse}(o, s) \mid o \sim p_T(o \mid I \oplus s), s \in S\}$

- I is a specific template/instruction for the teacher LLM
- D_I^{kd} is the teacher knowledge examples
- The **Parse** function parses a generation from the model into a knowledge example
- p_T is the teacher LLM generating distribution
- S is the set of seed knowledge examples

人话直觉

这页的公式看起来抽象，其实就是一句话：给 teacher 一个“题型说明” I ，再塞进一个 seed s ，teacher 按自己的生成分布吐出原始输出 o ；但 o 通常是一段文本，不一定直接能训练，所以还要 **Parse** 成结构化样本。**Parse** 就像把老师手写的批注整理成 CSV：不整理，student 训练时吃不进去。

精确定义 I 机制：数据获取阶段写作 $D_I^{kd} = \{\text{Parse}(o, s) \mid o \sim p_T(o \mid I \oplus s), s \in S\}$ 。其中 I 是 teacher 的 template 或 instruction； S 是 seed knowledge 集合； $I \oplus s$ 表示把 instruction 与 seed 组合成 teacher input； p_T 是 teacher LLM 的生成分布； o 是 teacher 生成的原始输出； $\text{Parse}(o, s)$ 把原始输出和 seed 解析成训练样本，例如 input-output pair、带解释的答案、偏好样本或反馈记录。随后 student 训练目标可写作 $L = \sum_I L_I(D_I^{kd}; \theta_S)$ ，表示可能对多个 templates 产生的数据分别定义训练损失，并优化 student 参数 θ_S 。

$$D_I^{kd} = \{\text{Parse}(o, s) \mid o \sim p_T(o \mid I \oplus s), s \in S\}, \quad L = \sum_I L_I(D_I^{kd}; \theta_S)$$

考试问法：公式题要逐项解释符号，不要只背公式。特别要说明 **Parse** 的作用：把 free-form generation 转成 structured distillation example。若问为什么有多个 I ，答不同 templates 覆盖不同技能、任务格式和领域，可增加数据多样性。

5. Labelling: 已有题目, 让 teacher 给答案

Approach 1: Labelling

Using teacher LLM to label the output y for a given input $x \in \mathcal{X}$ as the seed knowledge, with instruction I or demonstrations $c = \{(x_i, y_i) \mid i \leq m\}$.
Formally,

$$\mathcal{D}^{(lab)} = \{(x, y) \mid x \sim \mathcal{X}, y \sim p_T(y \mid I \oplus c \oplus x)\}$$

- Seed knowledge could be obtained from existing datasets
- The instruction prompt I or c often includes chain-of-thought to help the student LLM

人话直觉

Labelling 是最像传统监督学习的一种 KD。你手里已经有一堆题目 x , 但缺答案 y , 或者答案质量不够好; 于是请强 teacher 来批卷。Student 最后学到的不是 teacher 的内部思考, 而是 teacher 给出的输入到输出映射。它简单、实用、适合 black-box API, 所以很常考。

精确定义 / 机制: Labelling 使用 teacher LLM 为给定输入 $x \in X$ 生成输出 y 。Seed knowledge 可以来自已有 datasets, 也可以在 prompt 中加入 demonstrations $c = \{(x_i, y_i) \mid i \leq m\}$, 有时还包含 chain-of-thought 来提升 teacher 输出质量或让 student 学习推理过程。形式上, $\mathcal{D}^{(lab)} = \{(x, y) \mid x \sim X, y \sim p_T(y \mid I \oplus c \oplus x)\}$ 。它的优点是输入分布比较可控, 因为 x 来自已有数据; 适合无标签或弱标签数据补标。局限是仍依赖已有输入集合, 并且 teacher 生成的 label 可能错误、偏置或过度自信。

$$\mathcal{D}^{(lab)} = \{(x, y) \mid x \sim X, y \sim p_T(y \mid I \oplus c \oplus x)\}$$

考试问法: 如果考 Labelling, 抓住“ x 已给定, y 由 teacher 生成”。常见对比题会问 Labelling vs Expansion: Labelling 有现成 input; Expansion 连 input 也让 teacher 生成。

6. Expansion 与 Data Curation: 让 teacher 出题, 甚至从领域说明造题

Approach 2: Expansion

Using only demonstrations to generate samples x and then generating y for them. The seed knowledge is now demonstrations c
Formally,

$$\mathcal{D}^{(exp)} = \{(x, y) \mid x \sim p_T(x \mid I \oplus c), y \sim p_T(y \mid I \oplus x)\}$$

- Less constrained by the need for an existing dataset, can rely on a handful examples c
- Less concern about privacy of data – LLM generates everything almost
- Issue 1: Quality of data is highly dependent on teacher LLM
- Issue 2: Samples may often be similar to each other (less diversity in the dataset)

人话直觉

如果 Labelling 是“老师给已有题目写答案”，Expansion 就是“老师先仿照几道例题出新题，再自己写答案”。再往前一步，Data curation 甚至不需要具体例题，只给老师一段领域说明，比如“生成 Python 练习题和解法”，让它从主题出发合成数据。这很有生产力，但也危险：老师如果爱出同一种题，学生就会被训练成偏科。

精确定义 / 机制： Expansion 只依赖 demonstrations c ，先由 teacher 生成新的 input x ，再为 x 生成 output y : $\mathcal{D}^{(exp)} = \{(x, y) \mid x \sim p_T(x \mid I \oplus c), y \sim p_T(y \mid I \oplus x)\}$ 。它的优点是不需要大量现成数据，隐私风险也较低，因为样本基本由 LLM 合成；缺点是质量高度依赖 teacher，样本可能彼此相似、缺乏多样性。Data curation 则用 meta-information m ，例如领域解释、主题列表或任务说明，让 teacher 从头合成数据: $\mathcal{D}^{(cur)} = \{(x, y) \mid x \sim p_T(x \mid I \oplus m), y \sim p_T(y \mid I \oplus x)\}$ 。UltraChat 通过给 LLM 主题并生成 instructions/conversations；coding domain 中也可让 LLM 生成 Python exercises、solutions 和 snippets。

$$\mathcal{D}^{(exp)} = \{(x, y) \mid x \sim p_T(x \mid I \oplus c), y \sim p_T(y \mid I \oplus x)\}, \quad \mathcal{D}^{(cur)} = \{(x, y) \mid x \sim p_T(x \mid I \oplus m), y \sim p_T(y \mid I \oplus x)\}$$

考试问法：三者对比最常考：Labelling 给定 x 生成 y ；Expansion 用 demonstrations c 生成 x 再生成 y ；Data curation 用 meta-information m 从领域描述合成 x, y 。高分答案要加风险：teacher 错误、模式单一、diversity 不足、synthetic bias。

7. Feature 与 Feedback: 从看答案, 到看分布、看评语

Approach 4: Feature

Formally,

$$\mathcal{D}^{(\text{feat})} = \{(x, y, \phi_{\text{feat}}(x, y; \theta_T)) \mid x \sim \mathcal{X}, y \sim \mathcal{Y}\}$$

- Annotated some output set $\mathcal{Y}$ with the teacher LLM internal representations
- $\phi(\cdot; \theta_T)$ represents the operation of extracting feature knowledge (such as output distribution)
- Directly optimise student LLM using these internal representations

Food for Thought: What is the main difference we introduce now?

人话直觉

前几种方法像只看老师最后写的答案。Feature 方法更贪心：它想看老师脑子里的草稿纸，比如输出分布、hidden states 或 representations。Feedback 方法则像让学生先答，老师再批注：哪里错、哪个回答更好、怎样改。这两者是理解 black-box 和 white-box 区别的关键。

精确定义 / 机制： Feature 方法构造 $\mathcal{D}^{(\text{feat})} = \{(x, y, \phi_{\text{feat}}(x, y; \theta_T)) \mid x \sim X, y \sim Y\}$ ，其中 ϕ_{feat} 表示从 teacher 中提取 feature knowledge 的操作，例如输出分布或内部表示。优势是比 hard label 更细粒度，能传递 teacher 的不确定性和表示结构；缺点是通常需要 white-box access，很难用于强 proprietary LLM。Feedback 方法构造 $\mathcal{D}^{(\text{fb})} = \{(x, y, \phi_{\text{fb}}(x, y; \theta_T)) \mid x \sim X, y \sim p_S(y|x)\}$ ：student 先对输入生成回答，teacher 再提供 assessment、corrective information 或 preference feedback。它适合针对 student underperforms 的样本改进，也可生成偏好数据 (x, y_w, y_l) 。

$$\mathcal{D}^{(\text{feat})} = \{(x, y, \phi_{\text{feat}}(x, y; \theta_T)) \mid x \sim X, y \sim Y\}, \quad \mathcal{D}^{(\text{fb})} = \{(x, y, \phi_{\text{fb}}(x, y; \theta_T)) \mid x \sim X, y \sim p_S(y|x)\}$$

考试问法： 如果考 black-box vs white-box KD，Feature 是典型 white-box：要 teacher 内部分布或表示。Feedback 更适合 black-box/proprietary teacher：只要 teacher 能评价 student outputs。若考 Feature vs Feedback：前者抽 teacher 内部知识，后者让 teacher 批 student 的输出。

8. Self-knowledge: 模型自己当老师，但要防止自嗨循环

Approach 5: Feedback

Formally,

$$\mathcal{D}^{(fb)} = \{(x, y, \phi_{fb}(x, y; \theta_T)) \mid x \sim \mathcal{X}, y \sim p_S(y \mid x)\}$$

- p_S is the student LLM. Generate responses to dataset examples from the student LLM, and get feedback from the teacher LLM (assessment, corrective information)
- Can also be used to create preference datasets (Bai et al., 2022)
- The teacher LLM can also focus more on examples where the student LLM underperforms

人话直觉

Self-knowledge 很像学生写完作文后自己改、自己打分、再把修改版加入练习册。好处是便宜、可迭代，不必每次都请外部 teacher；坏处也明显：如果学生一开始就误解题目，它可能把错误润色得更像正确答案，然后一轮轮强化。

精确定义 / 机制：Self-knowledge 中，一个 LLM 同时扮演 teacher 和 student，通过生成、过滤、奖励、增强或评价自己的输出进行迭代改进。形式上， $\mathcal{D}^{(sk)} = \{(x, y, \phi_{sk}(x, y)) \mid x \sim S, y \sim p_S(y \mid I \oplus x)\}$ ，其中 ϕ_{sk} 可以是 filtering、rewarding、enhancing、evaluating 等附加操作。Self-Instruct 是典型例子：使用 GPT-3 做 Expansion，然后迭代扩大 seed data。Self-knowledge 的价值在于低成本自我扩增，但风险是放大自身错误、偏见、格式套路和低多样性，导致数据退化。

$$\mathcal{D}^{(sk)} = \{(x, y, \phi_{sk}(x, y)) \mid x \sim S, y \sim p_S(y \mid I \oplus x)\}$$

考试问法：如果考 Self-knowledge，要明确 teacher 和 student 是同一个模型，区别于 Feedback 中通常由 teacher 评价 student。答题时一定提 ϕ_{sk} 的作用：过滤、奖励、增强或评价，而不是只说“模型自己生成数据”。

9. 把知识转给 student: SFT、分布对齐、RL 与 DPO 各学什么

Approach 3: Reinforcement learning

A reminder of RL and how it would be applied here:

- Stage 1: Train a reward model r_ϕ using the feedback data $\mathcal{D}^{(fd)}$ generated by the teacher LLM
- Typical feedback data is preference data: (x, y_w, y_ℓ) , $y_w > y_\ell$
- Loss function for reward model is expected value over data of the log of the sigmoid of the difference between rewards (based on r_ϕ) for y_w and y_ℓ
- Stage 2: Train the student model to maximise the reward for its outputs (based on r_ϕ while minimising distance from the reference (original) model

人话直觉

拿到 teacher 产生的知识后，问题变成“怎么喂给 student”。SFT 像让学生反复背标准答案；divergence/similarity 像要求学生不只答案像，连概率分布或中间思路也像；RL 像先训练一个评分老师，再让学生朝高分回答优化；DPO 则更直接：告诉 student 哪个回答赢、哪个回答输，让它把赢家概率抬高。

精确定义 / 机制：讲义列出四类 transfer methods。Supervised finetuning 用 curated/distilled dataset 继续训练 student，最常见也最直观，但有 learning rate、epochs、format、data mix、overfitting 等 black art。Divergence/similarity 关注 white-box teacher: divergence 最小化 $E[D(p_T(y|x), p_S(y|x))]$ ，让 student 输出分布接近 teacher；similarity 最大化 $E[L_F(\Phi_T(f_T(x,y)), \Phi_S(f_S(x,y)))]$ ，让 teacher/student feature maps 在同形空间中相似，更常见于 encoder-based LMs。Reinforcement learning 先用 teacher feedback data 训练 reward model r_ϕ ，典型偏好数据为 (x, y_w, y_l) ，再训练 student 最大化 reward，同时控制与 reference/original model 的距离。Ranking optimisation 如 DPO 则直接基于 preference data 优化 student，相对 π_{ref} 提高 preferred response y_w 的概率、降低 rejected response y_l 的概率。

$$E[D(p_T(y|x), p_S(y|x))], \quad E_{(x, y_w, y_l) \sim D^{(fd)}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right]$$

考试问法：如果考“how to transfer knowledge to student”，列四类并各一句：SFT learns teacher-generated answers；divergence/similarity aligns distributions or representations and usually needs white-box access；RL trains a reward model from teacher feedback then optimizes student；DPO/ranking optimisation directly uses preference pairs。若问 RL vs DPO：RL 先显式训练 reward model，DPO 直接优化偏好概率比。

考试速记版

- KD 的核心: teacher 生成数据、标签、反馈、偏好或分布, student 再用这些知识 further train。
- Proprietary LLM 强但 black-box、贵、有 privacy issue; open-source LLM 可控、便宜、可本地部署但通常能力弱。
- LLM KD 的三类角色: capability enhancement、compression for efficiency、self-improvement via self-generated knowledge。
- KD recipe: target skill/domain + template $I \rightarrow$ seed knowledge $S \rightarrow$ teacher generation $\rightarrow$ train student。
- $D_{I^{kd}}$ 公式里, p_T 是 teacher generation distribution, Parse 把原始输出 o 变成结构化训练样本。
- Labelling: x 已有, teacher 生成 y ; Expansion: teacher 生成 x 再生成 y ; Data curation: 根据 meta-information m 合成数据。
- Feature 方法使用 teacher 内部表示或输出分布, 通常需要 white-box access。
- Feedback 方法让 student 先生成 y , 再由 teacher 评价、纠正或产生 preference data。
- Self-knowledge 中同一个模型既当 student 又当 teacher, 通过过滤、奖励、增强或评价自身输出迭代改进。
- SFT 学 teacher 生成的离散答案; divergence/similarity 对齐分布或表示。
- RL 用 teacher feedback 训练 reward model, 再优化 student; DPO 直接用 preference pairs 做 ranking optimisation。
- Black-box KD 更适合 proprietary teacher: labelling、expansion、data curation、feedback; white-box KD 包括 feature、divergence、similarity。

一段稳妥考场回答

Knowledge Distillation in LLMs is a framework for transferring knowledge from a stronger teacher model to a cheaper, smaller or more controllable student model. It is motivated by the contrast between proprietary LLMs, which are robust and high-performing but black-box, costly and privacy-sensitive, and open-source LLMs, which are controllable and locally deployable but often weaker. A typical KD recipe first chooses a target skill or domain and a template I , then prepares seed knowledge S , uses the teacher distribution p_T to generate raw outputs, parses them into distillation examples D_I^{kd} , and finally trains the student with these examples. Knowledge can be extracted through labelling, where x is given and teacher generates y ; expansion, where teacher generates both new x and y from demonstrations; data curation, where meta-information is used to synthesize data; feature methods, which use teacher internal distributions or representations and therefore usually need white-box access; feedback, where teacher evaluates student outputs; and self-knowledge, where a model iteratively filters or improves its own generations. After obtaining the knowledge, it can be transferred by supervised fine-tuning, divergence or similarity matching, reinforcement learning from teacher-generated feedback, or ranking optimisation such as DPO. The key exam distinction is that black-box KD can use teacher outputs or judgements from an API, while white-box KD requires access to internal representations or full distributions; in all cases, data quality, diversity and inherited teacher errors are central risks.